

GreenOps Platform — Architecture Document

Table of Contents

1. Project Overview	1
1.1. Objectives	1
2. Global Architecture	2
2.1. Architecture Diagram	2
2.2. Request Flow	3
3. Component Roles	3
3.1. Postgres	3
3.2. Keycloak	4
3.3. Frontend	4
3.4. alerts-service	4
3.5. user-service	5
3.6. Nginx	5
3.7. nginx-exporter	6
3.8. Prometheus	6
3.9. Grafana	7
3.9.1. Auto-provisioning	7
3.9.2. Dashboard: GreenOps — Service Overview	7
3.10. Redis	8
3.11. postgres-exporter	9
4. Docker Infrastructure	9
4.1. Prerequisites	9
4.2. Hosts File	9
4.3. Access Points	10
4.4. Default Credentials	10
4.5. Useful Commands	10
5. Configuration	10
5.1. Environment Variables	11
5.2. Database Schemas	11
6. CI/CD Pipelines	12
6.1. CI — Code Quality	12
6.2. Publish — Docker Images	13
7. Repository Layout	14
8. Kubernetes (Minikube)	15

8.1. Architecture	15
8.2. Namespace	15
8.3. Deployments	15
8.4. Services	16
8.5. ConfigMaps	16
8.6. Secrets	17
8.6.1. TLS Certificates	17
8.7. Storage	17
8.8. Keycloak CORS	17
8.9. Deployment	18
8.9.1. Post-deployment	18
8.9.2. Verification	19
8.10. Differences from Docker	20
8.11. Default Credentials	20

Authors: Marco KUIDJA, Moussa TOURE, Wilfried MAILLET Version: 1.0 Date: 2026-06

1. Project Overview

GreenOps is a cloud-native SaaS platform that monitors and visualizes energy consumption metrics. It demonstrates modern DevOps practices:

- Microservices architecture
- Docker containerization
- Kubernetes orchestration on Minikube
- Observability with Prometheus + Grafana
- Centralized authentication via Keycloak
- CI/CD with GitHub Actions
- Multi-schema PostgreSQL database

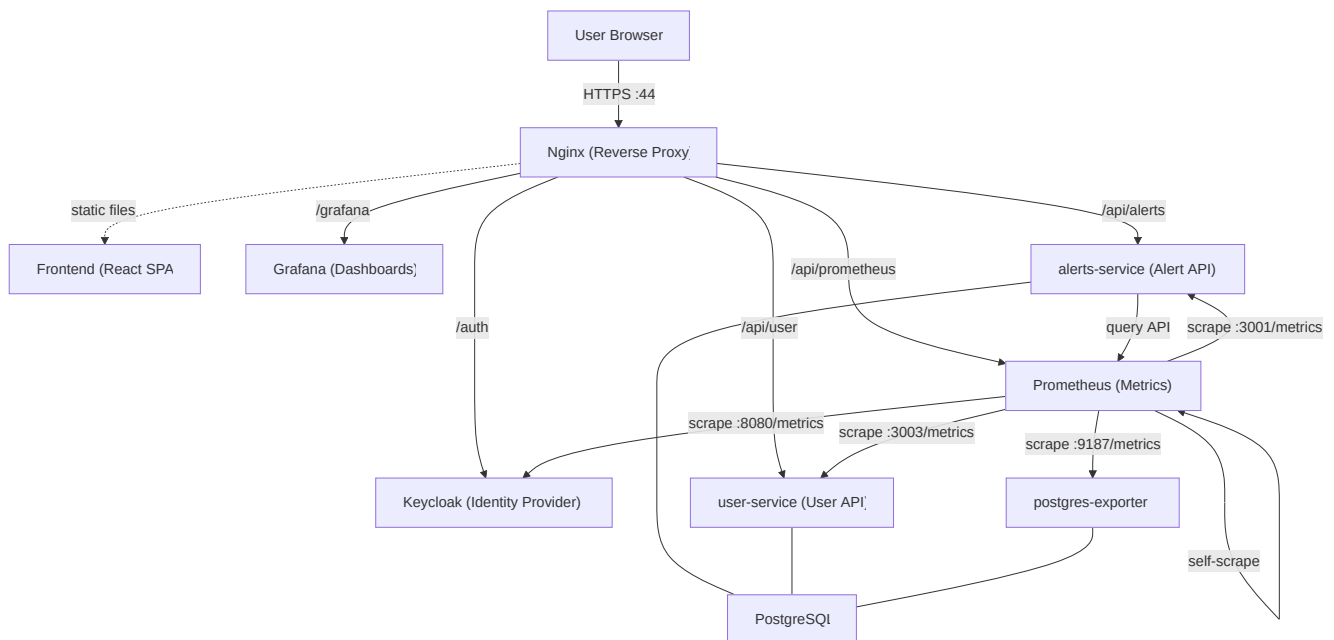
1.1. Objectives

- Collect and display energy metrics
- Manage user profiles and preferences
- Generate alerts based on Prometheus queries
- Provide a production-ready, containerized infrastructure
- Enable monitoring and observability out of the box

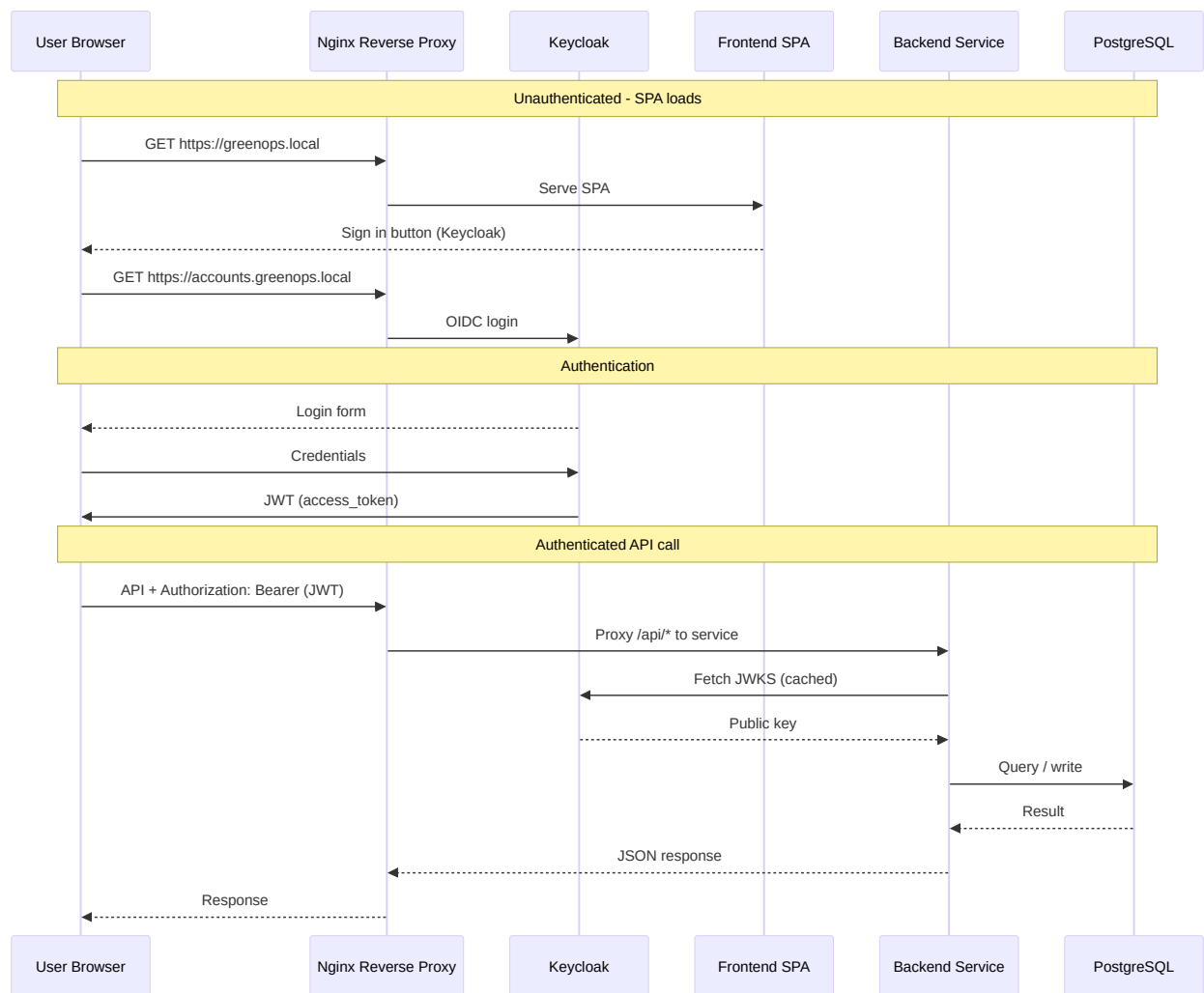
2. Global Architecture

Frontend	React SPA served by Nginx
API Gateway	Nginx reverse proxy (single entry point)
Identity Provider	Keycloak (OIDC / JWT)
Backend Services	alerts-service, user-service (Node.js + Express)
Database	PostgreSQL (multi-schema)
Monitoring	Prometheus (metrics), Grafana (dashboards)

2.1. Architecture Diagram



2.2. Request Flow



Route mapping from Nginx to backend services:

- `/api/alerts/*` → `alerts-service:3001`
- `/api/user/` → `user-service:3003/profile/`
- `/api/prometheus/*` → `prometheus:9090`

3. Component Roles

3.1. Postgres

Image `postgres:16-alpine`

Port `5432` (internal only)

Purpose Central database for all services.

Uses a multi-schema design within a single database:

schema `users` — `UserProfile` table (user-service)

- schema `alerts` — `Alert` table (alerts-service)

Persistent volume `postgres_data` ensures data survives restarts.

3.2. Keycloak

Image `quay.io/keycloak/keycloak:24.0`

Port `8080` (internal only)

Purpose OpenID Connect (OIDC) identity provider.

Key features:

- Exposed via Nginx at <https://accounts.greenops.local>.
- Pre-configured realm `greenops` imported from `infrastructure/docker/realm.json`.
- Clients: `greenops-frontend` (public) and `greenops-grafana` (confidential).
- Metrics enabled (`KC_METRICS_ENABLED=true`) for Prometheus scraping.
- JWT signing keys are fetched by backends via the JWKS URI.

3.3. Frontend

Build Vite + React 19 (TypeScript)

Docker `apps/frontend/Dockerfile` (multi-stage, served by Nginx)

The SPA is built at image creation time and served as static files by Nginx. It uses `react-oidc-context` for Keycloak authentication and Axios for API calls. The JWT is automatically attached by a request interceptor.

Pages:

- **Dashboard** — shows alert count, theme, Prometheus scrape durations, and recent alerts in a table.
- **Services** — displays live up/down status of all infrastructure components by querying the Prometheus `up` metric.

3.4. alerts-service

Language Node.js + Express 5 (TypeScript)

Port `3001`

Dockerfile `apps/alerts-service/Dockerfile`

REST API for managing alerts. Exposes the following endpoints:

- `GET /alerts` — list all alerts (most recent first)
- `POST /alerts` — create a new alert

- `PATCH /alerts/:id/read` — mark an alert as read
- `GET /health` — health check
- `GET /metrics` — Prometheus metrics (via `prom-client`)

It also runs a background monitor (`src/monitor.ts`) that:

1. Queries Prometheus every 30 seconds for configured alert rules.
2. If a threshold is breached, creates an `Alert` record in PostgreSQL.
3. When the metric recovers, marks pending alerts as read.

Alert rules include service-down detection and high scrape duration warnings.

3.5. user-service

Language Node.js + Express 5 (TypeScript)

Port 3003

Dockerfile `apps/user-service/Dockerfile`

REST API for user profiles. Exposes the following endpoints:

- `GET /profile` — fetch or auto-create the authenticated user's profile
- `PATCH /profile` — update profile fields (theme, notifications, etc.)
- `GET /health` — health check
- `GET /metrics` — Prometheus metrics (via `prom-client`)

Profiles are keyed by the `sub` claim from the JWT (Keycloak user ID). If a profile is not found, the service attempts a lookup by email first to handle Keycloak user-ID changes gracefully.

3.6. Nginx

Image `nginx:alpine` (for the SPA) + config from `infrastructure/docker/nginx.conf`

Ports 80:80, 443:443 (+ internal 8081 for `stub_status`)

Serves four responsibilities:

- **Static file server** — serves the built React SPA at `/`.
- **Reverse proxy** — routes API paths to backend services.
- **TLS termination** — terminates HTTPS with self-signed certificates.
- **stub_status** — exposes connection metrics on port 8081 for `nginx-exporter`.

Four server blocks (three virtual hosts + one internal):

Listen	Server Name	Purpose
8081	localhost	stub_status (internal, for nginx-exporter)
80/443	greenops.local	Frontend + API
80/443	accounts.greenops.local	Keycloak
80/443	monitor.greenops.local	Grafana

3.7. nginx-exporter

Image `nginx/nginx-prometheus-exporter:latest`

Port `9113` (internal)

Scrape URI `http://nginx:8081/stub_status`

Sidecar/exporter that reads Nginx stub_status and exposes Prometheus-formatted metrics at `/metrics`. In Docker Compose it runs as a separate service; in Kubernetes it runs as a sidecar container in the frontend pod.

3.8. Prometheus

Image `prom/prometheus`

Port `9090` (internal only)

Config `infrastructure/docker/prometheus.yml`

Scrapes the following targets every 15 seconds:

Job	Target	Metrics
prometheus	localhost:9090	Self-metrics
alerts-service	alerts-service:3001	Node.js (CPU, memory, event loop, handles)
user-service	user-service:3003	Node.js (CPU, memory, event loop, handles)
postgres-exporter	postgres-exporter:9187	PostgreSQL (connections, size, cache)
keycloak	keycloak:8080/metrics	JVM (heap, threads, GC, CPU)
nginx	nginx-exporter:9113	Nginx connections, request rate

The frontend queries Prometheus through the Nginx proxy for:

`scrape_duration_seconds` — displayed as bar charts on the Dashboard.

- `up` — used by the Services page for live status indicators.

3.9. Grafana

Image `grafana/grafana`

Port `3000` (internal only)

Auth OIDC via Keycloak (`greenops-grafana` client)

Configured with generic OAuth using Keycloak as the provider. Grafana is accessed at <https://monitor.greenops.local>.

3.9.1. Auto-provisioning

Grafana is fully auto-configured on startup — no manual setup required:

Config	Source
Prometheus datasource	<code>infrastructure/grafana/provisioning/datasources/datasource.yml</code>
Dashboard loader	<code>infrastructure/grafana/provisioning/dashboards/dashboard-provider.yml</code>
Dashboard JSON	<code>infrastructure/grafana/dashboards/greenops-overview.json</code>

3.9.2. Dashboard: GreenOps — Service Overview

The provisioned dashboard contains 12 panels:

#	Panel	Key Query	Source
1	Service Health	<code>up{job=~"\$job"}</code>	All targets
2	Memory Usage	<code>process_resident_memory_bytes</code>	Node.js
3	Heap Utilization	<code>nodejs_heap_size_used_bytes / nodejs_heap_size_total_bytes</code>	Node.js
4	CPU Usage	<code>rate(process_cpu_user_seconds_total[1m])</code>	Node.js
5	Event Loop Lag	<code>nodejs_eventloop_lag_seconds</code>	Node.js
6	Active Requests	<code>nodejs_active_requests_total</code>	Node.js
7	Postgres Overview	<code>pg_stat_activity_count , pg_database_size_bytes</code>	PostgreSQL
8	Scrape Duration	<code>scrape_duration_seconds (table)</code>	Prometheus
9	Nginx Connections	<code>nginx_connections_active / reading / writing / waiting</code>	Nginx
10	Nginx Request Rate	<code>rate(nginx_http_requests_total[1m])</code>	Nginx
11	Keycloak JVM Heap	<code>jvm_memory_used_bytes , jvm_memory_max_bytes</code>	Keycloak
12	Keycloak CPU & Threads	<code>system_cpu_usage , jvm_threads_live_threads</code>	Keycloak

3.10. Redis

Image `redis:7-alpine`

Port `6379` (internal only)

Purpose Caching, rate limiting, and alert deduplication.

Three use cases:

- **Profile cache** (user-service) — caches user profiles for 5 minutes; invalidated on update.
- **Rate limiting** (both services) — 100 requests/minute per client using `express-rate-limit` + `rate-limit-redis`.
- **Alert dedup** (alerts-service monitor) — prevents duplicate alert creation within configurable TTLs via `SET NX EX` (critical=10min, warning=5min, info=2min).

The Redis client (`ioredis`) uses `lazyConnect: true` so the server starts before Redis is available. Connection failures are logged but don't block the service.

3.11. postgres-exporter

Image prometheuscommunity/postgres-exporter:latest

Port 9187 (internal only)

Purpose Exposes PostgreSQL metrics to Prometheus.

4. Docker Infrastructure

4.1. Prerequisites

- Docker Engine 24+ and Docker Compose v2
- openssl (to generate certificates)
- pnpm (for local development outside Docker)

Run the following from the repository root:

```
# 1. Generate self-signed TLS certificates (one time only)
mkdir -p infrastructure/docker/certs
openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
  -keyout infrastructure/docker/certs/nginx.key \
  -out infrastructure/docker/certs/nginx.crt \
  -subj "/CN=greenops.local" \
  -addext
"subjectAltName=DNS:greenops.local,DNS:accounts.greenops.local,DNS:monitor.greenops.local"

# 2. Start all services
docker compose -f infrastructure/docker/compose.yml up -d --build

# 3. Check that everything is running
docker compose -f infrastructure/docker/compose.yml ps

# 4. Follow logs
docker compose -f infrastructure/docker/compose.yml logs -f
```

4.2. Hosts File

Add the following entries to `/etc/hosts`:

```
127.0.0.1 greenops.local
127.0.0.1 accounts.greenops.local
127.0.0.1 monitor.greenops.local
```

4.3. Access Points

Service	URL
Frontend	https://greenops.local
Keycloak	https://accounts.greenops.local
Grafana	https://monitor.greenops.local

4.4. Default Credentials

Service	Username	Password
Keycloak	admin	admin
Grafana	OIDC	(login via Keycloak)

4.5. Useful Commands

```
# Rebuild a single service
docker compose -f infrastructure/docker/compose.yml up -d --build <service>

# View logs for a service
docker compose -f infrastructure/docker/compose.yml logs -f <service>

# Stop everything
docker compose -f infrastructure/docker/compose.yml down

# Stop and remove volumes (fresh start)
docker compose -f infrastructure/docker/compose.yml down -v

# Run a command inside a container
docker compose -f infrastructure/docker/compose.yml exec <service> <command>
```

5. Configuration

5.1. Environment Variables

Variable	Default	Service(s)
DATABASE_URL	postgresql://greenops:...	alerts-service, user-service
PORT	3001 / 3003	alerts-service, user-service
KEYCLOAK_JWKS_URI	http://keycloak:8080/...	alerts-service, user-service
PROMETHEUS_URL	http://prometheus:9090	alerts-service
REDIS_URL	redis://:password@redis:6379	alerts-service, user-service
REDIS_HOST	redis	alerts-service, user-service (K8s)
REDIS_PORT	6379	alerts-service, user-service (K8s)
REDIS_PASSWORD	<i>(from secret)</i>	alerts-service, user-service (K8s)

5.2. Database Schemas

Both services connect to the same `greenops_db` database using distinct PostgreSQL schemas:

```

-- Schema: users (user-service)
CREATE SCHEMA IF NOT EXISTS users;

CREATE TABLE users."UserProfile" (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  "keycloakId" TEXT NOT NULL UNIQUE,
  email       TEXT NOT NULL UNIQUE,
  theme       TEXT NOT NULL DEFAULT 'light',
  notifications BOOLEAN NOT NULL DEFAULT true,
  "dashboardLayout" JSONB,
  "createdAt" TIMESTAMPTZ NOT NULL DEFAULT now(),
  "updatedAt" TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Schema: alerts (alerts-service)
CREATE SCHEMA IF NOT EXISTS alerts;

CREATE TYPE alerts."Severity" AS ENUM ('info', 'warning', 'critical');

CREATE TABLE alerts."Alert" (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  title       TEXT NOT NULL,
  description  TEXT NOT NULL,
  severity    alerts."Severity" NOT NULL DEFAULT 'info',
  "metricName" TEXT NOT NULL,
  "metricValue" DOUBLE PRECISION NOT NULL,
  threshold   DOUBLE PRECISION NOT NULL,
  "isRead"    BOOLEAN NOT NULL DEFAULT false,
  "createdAt" TIMESTAMPTZ NOT NULL DEFAULT now()
);

```

6. CI/CD Pipelines

6.1. CI — Code Quality

Triggered on push / pull request to `main` or `master`.

1. `pnpm install --frozen-lockfile`
2. `pnpm biome ci .` — lint and format checks
3. `pnpm --filter shared run build` — compile the shared types package
4. `pnpm --filter alerts-service exec prisma generate && pnpm --filter alerts-service exec tsc --noEmit` — type-check alerts-service
5. `pnpm --filter user-service exec prisma generate && pnpm --filter user-service exec tsc --noEmit` — type-check user-service
6. `pnpm --filter frontend exec tsc --noEmit` — type-check frontend



alerts-service and user-service share the same `@prisma/client` in the pnpm store. Each `prisma generate` overwrites the other's models, so they must be type-checked sequentially, regenerating Prisma immediately before each check.

6.2. Publish — Docker Images

Triggered on push to `main` / `master` or tag matching `v*`.

Publishes three images to GitHub Container Registry (GHCR):

Image	Dockerfile
<code>ghcr.io/<org>/green-ops/frontend</code>	<code>apps/frontend/Dockerfile</code>
<code>ghcr.io/<org>/green-ops/alerts-service</code>	<code>apps/alerts-service/Dockerfile</code>
<code>ghcr.io/<org>/green-ops/user-service</code>	<code>apps/user-service/Dockerfile</code>

Tags: `latest` (branch), `sha-<short>` (always), semver (tags).

7. Repository Layout

```
.
├── .github/workflows/
│   ├── ci.yml                # Lint, type-check
│   └── publish.yml           # Build & push Docker images to GHCR
├── apps/
│   ├── alerts-service/      # Alert REST API + Prometheus monitor
│   ├── frontend/           # React SPA (Vite)
│   └── user-service/        # User profile REST API
├── infrastructure/
│   ├── docker/
│   │   ├── certs/          # TLS certificates (gitignored)
│   │   ├── compose.yml     # Docker Compose definition
│   │   ├── nginx.conf      # Nginx reverse proxy config (Docker)
│   │   ├── prometheus.yml  # Prometheus scrape config (Docker)
│   │   └── realm.json      # Keycloak realm import (Docker)
│   ├── grafana/
│   │   ├── dashboards/
│   │   │   └── greenops-overview.json # Provisioned dashboard (12 panels)
│   │   └── provisioning/
│   │       ├── datasources/
│   │       │   └── datasource.yml     # Prometheus datasource
│   │       └── dashboards/
│   │           └── dashboard-provider.yml # Dashboard loader
│   └── k8s/
│       ├── 00-namespace.yml # greenops namespace
│       ├── 01-secrets.yml   # Secret definitions (values from deploy.sh)
│       ├── 02-configmaps.yml # Placeholder (ConfigMaps created by deploy.sh)
│       ├── 03-postgres.yml  # PostgreSQL + postgres-exporter
│       ├── 04-keycloak.yml  # Keycloak (+ CORS config)
│       ├── 05-prometheus.yml # Prometheus
│       ├── 06-grafana.yml   # Grafana + provisioning volume mounts
│       ├── 07-alerts-service.yml # Alert API
│       ├── 08-user-service.yml # User profile API
│       ├── 09-frontend.yml  # Frontend (nginx + nginx-exporter sidecar)
│       ├── 10-redis.yml     # Redis (PVC + Deployment + Service)
│       ├── deploy.sh        # One-command Minikube deployment
│       ├── nginx.conf       # Nginx config (K8s version, includes stub_status)
│       ├── prometheus.yml   # Prometheus config (K8s version, includes nginx target)
│       └── realm.json       # Keycloak realm import (K8s version)
├── packages/
│   └── shared/              # Shared Zod schemas, JWT auth middleware
├── docs/
│   └── architecture.adoc   # This document
├── pnpm-workspace.yaml     # Monorepo workspace config
├── package.json            # Root scripts (lint, build, type-check)
├── tsconfig.json          # Base TypeScript config
└── biome.json              # Biome (linter + formatter) config
```

8. Kubernetes (Minikube)

The project includes a complete Kubernetes deployment for Minikube under `infrastructure/k8s/`. It mirrors the Docker Compose architecture and uses the same remote images from GHCR.

8.1. Architecture

In the K8s setup, the Nginx reverse proxy runs as a container in the `frontend` pod (same image as Docker), with its config from `infrastructure/k8s/nginx.conf` mounted as a ConfigMap. The LoadBalancer service exposes it externally.

8.2. Namespace

All resources are deployed in a single namespace:

```
greenops
```

8.3. Deployments

Deployment	Image	Port(s)	Readiness Probe
frontend	ghcr.io/.../frontend:latest	443, 80	—
<i>nginx-exporter</i>	<i>nginx/nginx-prometheus-exporter</i>	<i>9113</i>	<i>(sidecar)</i>
alerts-service	ghcr.io/.../alerts-service:latest	3001	GET /health
user-service	ghcr.io/.../user-service:latest	3003	GET /health
postgres	postgres:16-alpine	5432	pg_isready
postgres-exporter	prometheuscommunity/postgres-exporter	9187	—
keycloak	quay.io/keycloak/keycloak:24.0	8080	—
redis	redis:7-alpine	6379	redis-cli ping
prometheus	prom/prometheus	9090	—
grafana	grafana/grafana	3000	—

8.4. Services

Service	Type	Port(s)	Purpose
frontend	LoadBalancer	443, 80, 9113	External entry + nginx metrics
alerts-service	ClusterIP	3001	Internal API
user-service	ClusterIP	3003	Internal API
postgres	ClusterIP	5432	Database
postgres-exporter	ClusterIP	9187	Metrics
keycloak	ClusterIP	8080	OIDC provider
redis	ClusterIP	6379	Cache / Rate limiting
prometheus	ClusterIP	9090	Metrics
grafana	ClusterIP	3000	Dashboards

The `frontend` LoadBalancer is the sole external entry point. `minikube tunnel` routes the LoadBalancer IP to the host.

8.5. ConfigMaps

ConfigMaps are created from standalone K8s config files in `infrastructure/k8s/` (separate from the Docker versions in `infrastructure/docker/`):

ConfigMap	Source File	Mount Path
nginx-config	<code>infrastructure/k8s/nginx.conf</code>	<code>/etc/nginx/conf.d/default</code>
prometheus-config	<code>infrastructure/k8s/prometheus.yml</code>	<code>/etc/prometheus/prometheu</code>
keycloak-realm	<code>infrastructure/k8s/realm.json</code>	<code>/opt/keycloak/data/import</code>
grafana-datasource	<code>infrastructure/grafana/provisioning/datasources/datasource.yml</code>	<code>/etc/grafana/provisioning</code>
grafana-dashboard-provider	<code>infrastructure/grafana/provisioning/dashboards/dashboard-provider.yml</code>	<code>/etc/grafana/provisioning provider.yml</code>
grafana-dashboard	<code>infrastructure/grafana/dashboards/greenops-overview.json</code>	<code>/etc/grafana/dashboards/g</code>

The nginx config (`infrastructure/k8s/nginx.conf`) routes by virtual host:

Hostname	Route	Backend
greenops.local	/api/alerts/*	alerts-service:3001
greenops.local	/api/user/*	user-service:3003
greenops.local	/api/prometheus/*	prometheus:9090
greenops.local	/ (static files)	frontend SPA
accounts.greenops.local	/	keycloak:8080
monitor.greenops.local	/	grafana:3000

8.6. Secrets

Secret	Contents
greenops-secrets	postgres-user, postgres-password, postgres-db, redis-password, grafana-client-secret
nginx-tls	nginx.crt, nginx.key (self-signed, generated at deploy time)

8.6.1. TLS Certificates

Unlike Docker (which persists certs in `infrastructure/docker/certs/`), K8s generates self-signed certificates at deploy time using `openssl` and stores them in a `kubernetes.io/tls` Secret. The certificates are never committed to the repository.

8.7. Storage

PostgreSQL uses a PersistentVolumeClaim:

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: postgres-pvc
  namespace: greenops
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
```

8.8. Keycloak CORS

The frontend SPA (<https://greenops.local>) makes OIDC discovery requests to Keycloak (<https://accounts.greenops.local>), which is a cross-origin request. Keycloak is configured with CORS support:

```
env:
- name: KC_HTTP_CORS_ALLOWED_ORIGINS
  value: "https://greenops.local"
- name: KC_HTTP_CORS_ALLOWED_METHODS
  value: "GET,POST,PUT,PATCH,DELETE,OPTIONS"
- name: KC_HTTP_CORS_ALLOWED_HEADERS
  value: "Authorization,Content-Type,X-Requested-With,Origin"
- name: KC_HTTP_CORS_EXPOSED_HEADERS
  value: "WWW-Authenticate"
```

8.9. Deployment

The `deploy.sh` script handles the full deployment:

```
bash infrastructure/k8s/deploy.sh
```

What the script does:

1. Generates self-signed TLS certificates
2. Starts Minikube if needed (Docker driver, 2 CPU, 4 GB RAM)
3. Creates the `greenops` namespace
4. Creates ConfigMaps from `infrastructure/k8s/*` config files
5. Creates a TLS Secret for nginx
6. Applies all manifests in order (PostgreSQL → Keycloak → Prometheus → services → frontend)
7. Waits for all pods to reach `Ready` state
8. Prints connection instructions

8.9.1. Post-deployment

```
# Start the LoadBalancer tunnel
minikube tunnel

# Get the assigned LoadBalancer IP
kubectl get svc frontend -n greenops
# NAME          TYPE          CLUSTER-IP    EXTERNAL-IP    PORT(S)
# frontend      LoadBalancer  10.107.80.138  10.107.80.138  443:31109/TCP,80:31448/TCP

# Add to /etc/hosts (use the EXTERNAL-IP from above)
# 10.107.80.138  greenops.local
# 10.107.80.138  accounts.greenops.local
# 10.107.80.138  monitor.greenops.local
```

8.9.2. Verification

```
# Check all pods are running
kubectl get pods -n greenops

# Test the frontend SPA
curl -sk https://greenops.local/ | head -5
# <!DOCTYPE html><html lang="en">...

# Test API proxy (expects JWT – will return auth error)
curl -sk https://greenops.local/api/alerts/alerts
# {"success":false,"error":"Authorization header missing"}

# Test Prometheus through proxy
curl -sk https://greenops.local/api/prometheus/-/ready
# Prometheus Server is Ready.

# Test Keycloak OIDC discovery
curl -sk https://accounts.greenops.local/realms/greenops/.well-known/openid-configuration
# {"issuer":"https://accounts.greenops.local/realms/greenops",...}

# Test CORS preflight
curl -sk -X OPTIONS -H "Origin: https://greenops.local" \
  -H "Access-Control-Request-Method: GET" \
  https://accounts.greenops.local/realms/greenops/.well-known/openid-configuration
# Access-Control-Allow-Origin: https://greenops.local
```

8.10. Differences from Docker

Aspect	Docker	K8s
Config files	infrastructure/docker/	infrastructure/k8s/ (separate tracking)
Grafana provisioning	Mounted from infrastructure/grafana/	ConfigMaps from same source
Certificates	Persistent in certs/ (committed)	Generated at deploy time (not committed)
Entry point	compose.yml	deploy.sh
Frontend service	Port mapping in compose	LoadBalancer + minikube tunnel
Nginx metrics	Separate nginx-exporter service	Sidecar container in frontend pod
TLS termination	Nginx in frontend container	Same (config via ConfigMap)
Service discovery	Docker DNS (service name)	K8s DNS (same name, same namespace)
PostgreSQL data	Named volume postgres_data	PVC postgres-pvc (1 GiB)
Redis	redis:7-alpine service	PVC + Deployment + Service

8.11. Default Credentials

Service	Username	Password
Keycloak	admin	admin
Keycloak	marco.kuidja	user1password
Grafana	OIDC	(login via Keycloak)